Introduction to Large Language Models

LLMs and Prompt Engineering

Ram N Sangwan

Large Language Model

A Language Model is a **Probabilistic Model** of Text.

I wrote to the zoo to send me a pet. They sent me a _____

Word	lion	elephant	dog	cat	panther	alligator
Probability	0.1	0.1	0.3	0.2	0.05	0.02

The LLM gives a probability to every word in its vocabulary of appearing in the blank.

"**Large**" here Refer to Number of Parameters. No agreed upon threshold

LLMs – What you need to Know.

- LLM Architectures
  - What else can LLMs do?
- Prompting and Training
  - How do we affect the distribution over the vocabulary?
- Decoding
  - How do LLMs generate text using these distributions?

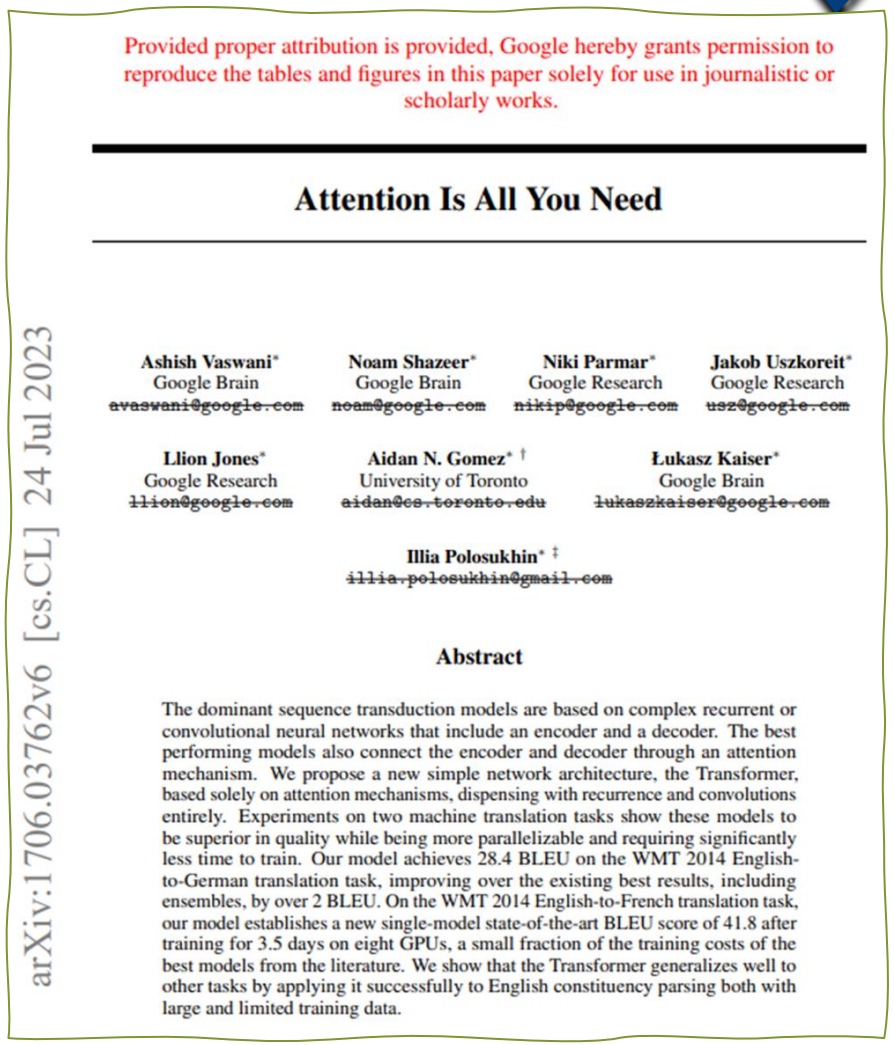
Encoders and Decoders

Multiple architectures focused on encoding and decoding, i.e., embedding and text generation

All Models built on the Transformer Architecture.

Each type of model has different capabilities (embedding / generation)

Models of each type come in a variety of sizes (# of parameters)



Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

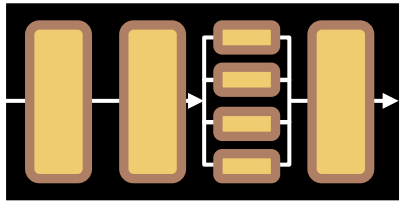
Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

arXiv:1706.03762v6 [cs.CL] 24 Jul 2023

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.



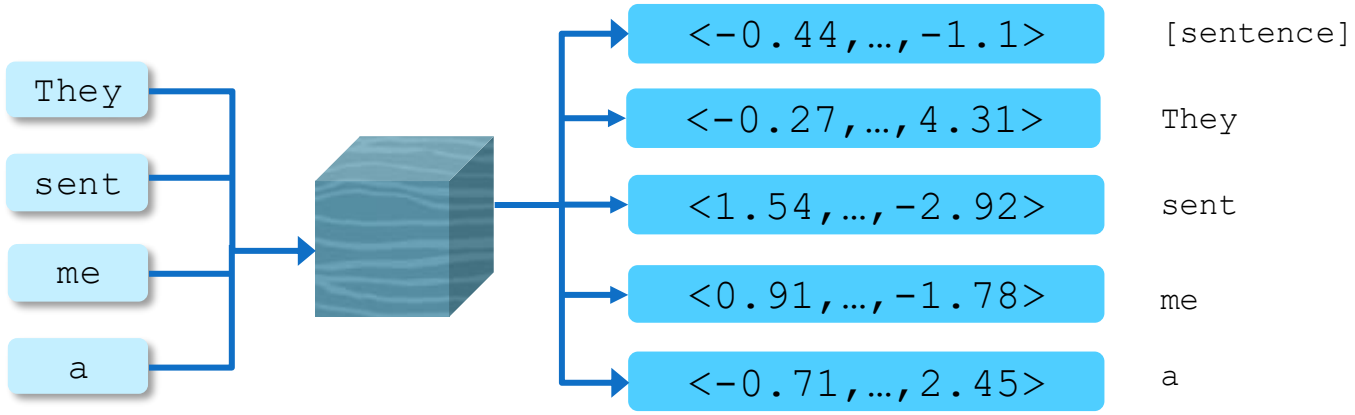
Transformers

Encoders

Encoder – models that convert a sequence of words to an embedding (vector representation)

Examples

Embed-light, BERT, RoBERTA, DistillBERT, SBERT,…

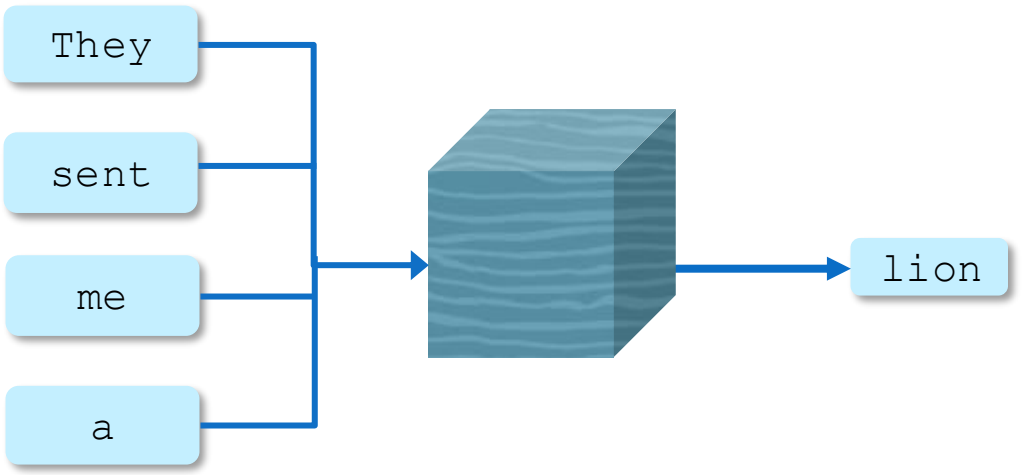


Primary uses: embedding tokens, sentences, & documents

Decoders

Decoder – models take a sequence of words and output next word

Examples
GPT-4, Llama, BLOOM, Falcon, …

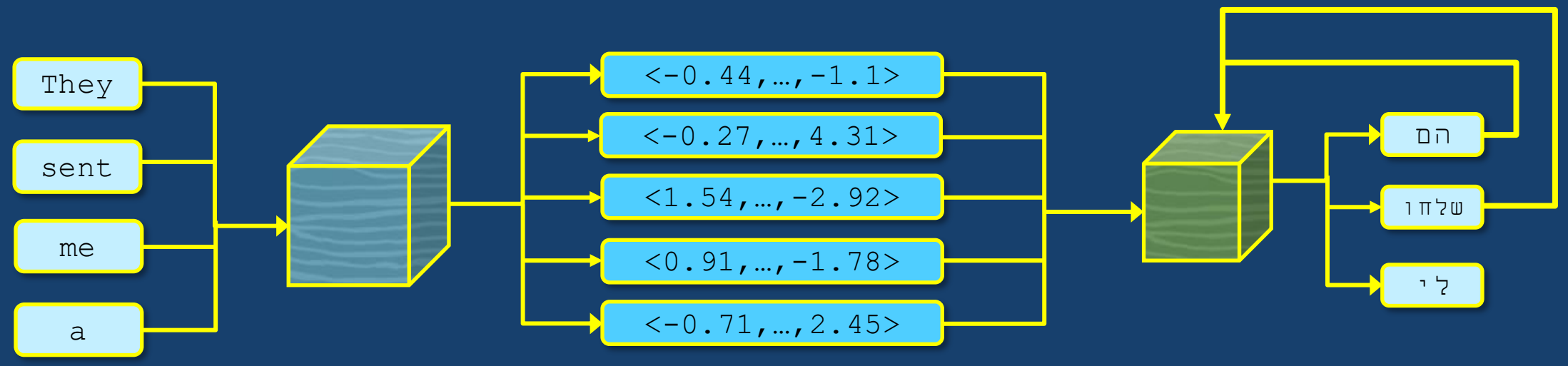


Primary uses: text generation, chat-style models, (including QA, etc…)

Encoders - Decoders

Encoder-decoder - encodes a sequence of words and use the encoding + to output a next word

Examples
T5, UL2, BART,…



Transformers Architecture

Transformers architecture eliminates the need for recurrent or convolutional layers

The Encoder stack contains multiple Encoders.

Each Encoder contains:

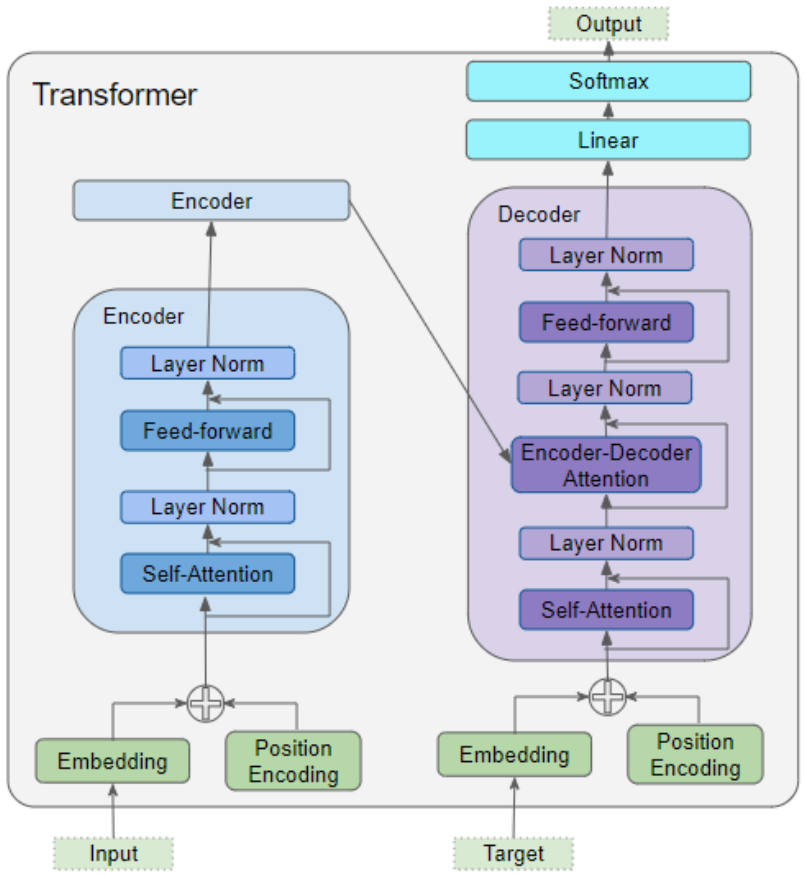
- Multi-Head Attention layer
- Feed-forward layer

The Decoder stack contains many Decoders. Each Decoder contains:

- Two Multi-Head Attention layers
- Feed-forward layer

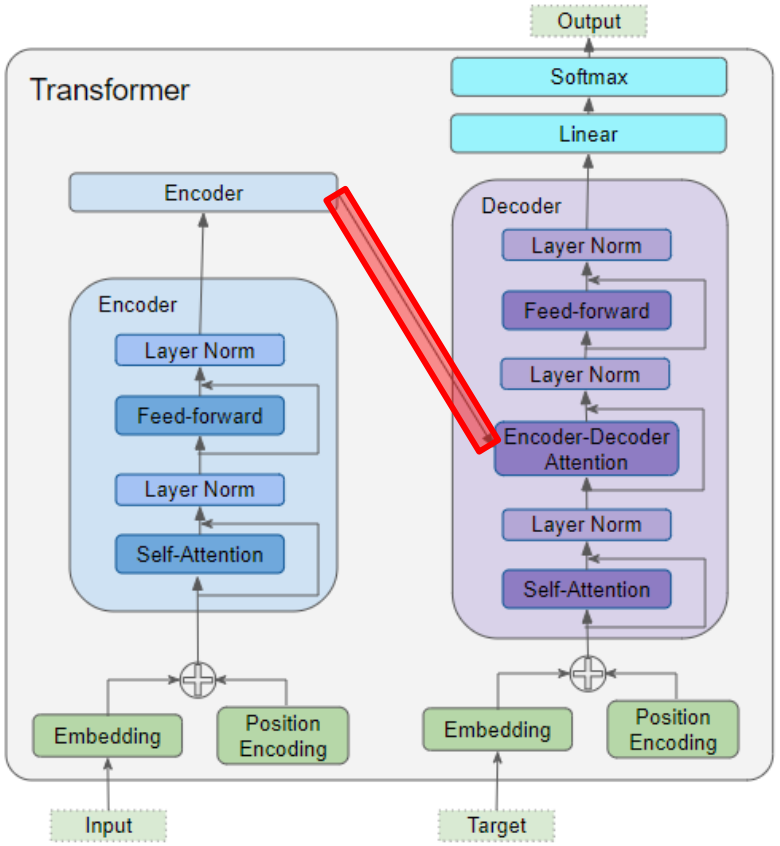
Output - generates the final output, and contains:

- Linear layer
- Softmax layer.



Transformers Architecture

- The data, that leaves the encoder, is *"a deep representation of the structure and meaning of the input sequences"*
- This representation is inserted into the middle of the decoder to influence decoder's self-attention mechanism.



Affecting the distribution over Vocabulary

I wrote to the zoo to send me a pet. They sent me a _____

Word	lion	elephant	dog	cat	panther	alligator
Probability	0.1	0.1	0.3	0.2	0.05	0.02

To exert some control over the LLM, we can affect the probability over vocabulary in 2 ways:

1. Prompting
2. Training

Aligning LLMs to Follow Instructions

- LLMs are trained to predict the next word on a large dataset of internet text, rather than to safely perform the language task that the user wants.
- Can not give instructions or ask questions to a LLM.
- Instead, formulate your input as a prompt whose natural continuation is your desired output.

Decoding

- The process of generating text with an LLM

```
I wrote to the zoo to send me a pet. They sent me a _______
```

Word	lion	elephant	dog	cat	panther	alligator
Probability	0.03	0.02	0.45	0.4	0.05	0.01

- Decoding happens iteratively, 1 word at a time
- At each step of decoding, we use the distribution over vocabulary and select 1 word to emit.
- The word is appended to the input, the decoding process continues.

Greedy Decoding

- Pick the highest probability word at each step

I wrote to the zoo to send me a pet. They sent me a _____

Word	lion	elephant	dog	cat	panther	alligator
Probability	0.03	0.02	0.45	0.4	0.05	0.01

I wrote to the zoo to send me a pet. They sent me a dog _____

Word	EOS	elephant	dog	cat	panther	alligator
Probability	0.99	0.02	0.45	0.4	0.05	0.01

Output: I wrote to the zoo to send me a pet. They sent me a dog.

Non-Deterministic Decoding

- Pick randomly among high probability candidates at each step.

I wrote to the zoo to send me a pet. They sent me a _____

Word	small	elephant	dog	cat	panda	alligator
Probability	0.01	0.02	0.25	0.4	0.05	0.01

I wrote to the zoo to send me a pet. They sent me a small_____

Word	small	elephant	dog	cat	panda	red
Probability	0.01	0.02	0.25	0.4	0.05	0.01

I wrote to the zoo to send me a pet. They sent me a small red_____

Word	small	elephant	dog	cat	panda	red
Probability	0.01	0.02	0.25	0.4	0.05	0.01

Output: I wrote to the zoo to send me a pet. They sent me a small red panda.

Training

- Prompting alone may be inappropriate when: training data exists, or domain adaptation is required.
- Domain-adaptation – adapting a model (typically via training) to enhance its performance outside of the domain/subject area it was trained on.

Training

Training Style	Modifies	Data	Summary
Fine-tuning (FT)	All parameters	Labeled, task-specific	Classic ML training
Param. Efficient FT	Few, new parameters	Labeled, task-specific	+Learnable params to LLM
Soft prompting	Few, new parameters	Labeled, task-specific	Learnable prompt params
(cont.) pre-training	All parameters	Unlabeled	Same as LLM pre-training

Hardware Cost

Model Size	Pre-train	Fine-tune	Prompt-tune	LORA	Inference
100M	8-16 GPUs 1 day	1 GPU hours	N/A	N/A	CPU / GPU
7B	512 GPUs 7 days *	8 GPUs hours-days	1 GPU hours	2 GPUs hours	1 GPU
65B	2048 GPUs 21 days *	48 GPUs 7 days	4 GPUs hours	16 GPUs hours	6 GPUs
170B	384 GPUs ~100 days **	100 GPUs weeks	48 GPUs hours-days	48 GPUs hours-days	8-16 GPUs

Thank You